

# Support de TP

Module DevSecOps — Node.js, Jest, Docker, GitLab CI/CD, Terraform

Auteur : Imane HADI



# Introduction

Ce support de travaux pratiques a pour objectif de guider les étudiants dans la réalisation d'une chaîne DevSecOps complète, depuis le développement d'une application Node.js jusqu'au déploiement multi-cloud sur AWS et Azure. Le TP couvre l'ensemble du cycle de vie logiciel : développement, tests automatisés, conteneurisation Docker, pipeline CI/CD GitLab avec contrôles de sécurité, publication dans un Container Registry, et déploiement infrastructure-as-code avec Terraform. L'approche multi-cloud (AWS pour le staging, Azure pour la production) permet de découvrir les spécificités de chaque fournisseur cloud tout en respectant les bonnes pratiques de séparation des environnements.

Ce document est structuré en parties progressives (A à I), chacune correspondant à une étape clé de la chaîne DevSecOps. Chaque partie inclut les objectifs, la procédure détaillée, le code source nécessaire, les points de vérification (checkpoints) et les erreurs fréquentes. Le respect de l'ordre des parties est essentiel car chaque étape construit sur la précédente.

## 1. Objectifs pédagogiques et résultats attendus

L'objectif principal de ce TP est de réaliser une chaîne DevSecOps complète : développement d'une application Node.js, tests automatisés, conteneurisation Docker, pipeline GitLab (qualité et sécurité), publication de l'image dans le GitLab Container Registry, création de l'infrastructure cloud avec Terraform (AWS pour le staging, Azure pour la production), et déploiement de l'image sur ces deux environnements. Ce TP permet de mettre en pratique les concepts fondamentaux du DevSecOps en intégrant la sécurité à chaque étape du cycle de développement.

### 1.1 Compétences visées

À l'issue du TP, l'étudiant sera en mesure de structurer une application Node.js/Express avec séparation des responsabilités (app/server) et gestion des variables d'environnement. Il maîtrisera l'écriture de tests automatisés avec Jest et Supertest, la conteneurisation Docker, la définition d'un pipeline GitLab CI/CD avec stages de test, sécurité, build et déploiement, ainsi que la description d'infrastructure cloud en Terraform et le déploiement d'images Docker depuis un registry privé.

- **Structurer une application Node.js/Express** : Séparation app/server, variables d'environnement, configuration PORT
- **Écrire des tests automatisés** : Jest et Supertest pour les routes HTTP, exécution en local et dans le pipeline CI
- **Conteneuriser avec Docker** : Dockerfile optimisé, .dockerignore, exploitation du GitLab Container Registry
- **Définir un pipeline CI/CD** : Stages test, security, build, deploy avec GitLab CI
- **Infrastructure as Code** : Terraform pour AWS EC2 et Azure App Service, déploiement d'images Docker
- **Comprendre le parcours de l'image** : Construction dans le pipeline, stockage dans le registry, récupération par AWS et Azure

### 1.2 Résultats attendus

Les résultats attendus à l'issue du TP sont les suivants : l'application doit être déployée et accessible (routes / et /health fonctionnelles en local, dans un conteneur Docker, sur l'environnement de staging AWS et sur l'environnement de production Azure). Le pipeline GitLab doit être vert (tests, sécurité, build, push registry). L'infrastructure doit être créée et détruite proprement avec Terraform.

## 2. Architecture cible de la solution

La solution comporte un dépôt GitLab contenant le code source et le fichier de pipeline ; un pipeline CI/CD qui exécute les tests, le contrôle de sécurité, le build de l'image Docker et sa publication dans le GitLab Container Registry ; deux environnements cloud créés par Terraform — staging sur AWS (instance EC2) et production sur Azure (App Service for Containers) — qui récupèrent l'image depuis le registry et exécutent le conteneur. Cette architecture illustre le principe de séparation des environnements : le staging permet de valider le comportement avant la mise en production, tandis que la production est l'environnement final exposé aux utilisateurs.

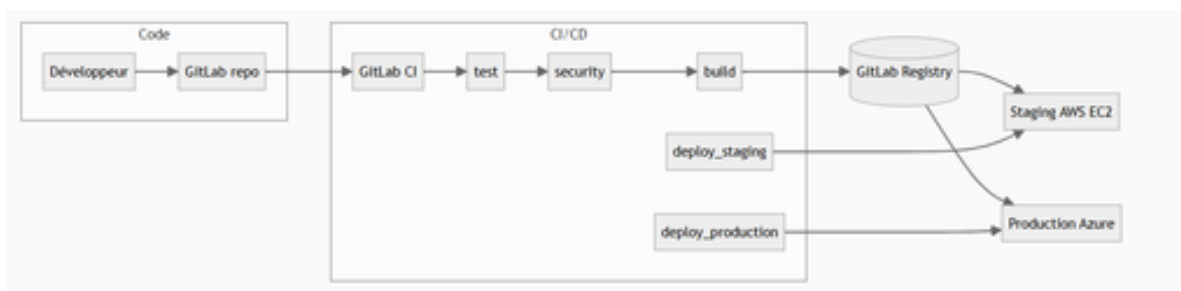


Figure 1 — Architecture DevSecOps globale

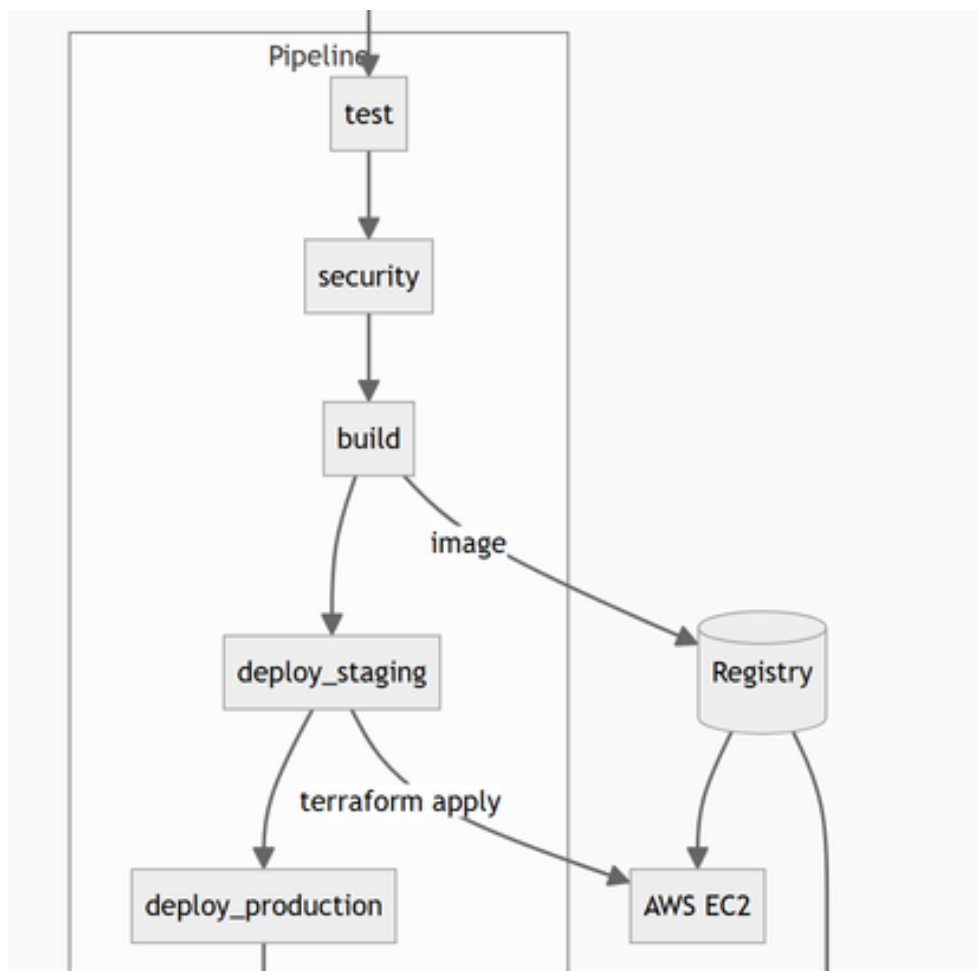


Figure 2 — Pipeline GitLab CI/CD (stages et flux)

## 2.1 Staging vs Production

Le staging (AWS) constitue l'environnement de préproduction où l'on déploie une version pour valider le comportement avant la mise en production. Il s'agit d'une instance EC2 qui exécute le conteneur Docker. La production (Azure) est l'environnement final exposé aux utilisateurs, matérialisé par une Web App Azure (App Service for Containers) qui tire l'image depuis le GitLab Registry et expose l'application.

## 2.2 Bonnes pratiques — Secrets

Il est crucial de ne jamais commiter de secrets (mots de passe, tokens) dans le code source. Pour les jobs CI/CD, il faut utiliser GitLab CI/CD Variables (Settings → CI/CD → Variables) avec l'option « Masked ». Pour Terraform, il convient d'utiliser des variables sensibles (sensitive = true) et des fichiers .tfvars non versionnés ou un backend distant pour le state. Ces pratiques garantissent que les informations d'identification ne sont jamais exposées dans le dépôt de code.

## 3. Préparation de l'environnement de travail

Avant de commencer le TP, il est essentiel de vérifier que tous les outils nécessaires sont installés et correctement configurés. La checklist suivante recense les éléments obligatoires et optionnels, ainsi que les commandes de vérification correspondantes.

### 3.1 Checklist logicielle

| Élément                   | Statut      | Vérification                   |
|---------------------------|-------------|--------------------------------|
| Node.js (LTS 20.x)        | Obligatoire | node -v                        |
| npm                       | Obligatoire | npm -v                         |
| Git                       | Obligatoire | git --version                  |
| Docker Desktop            | Obligatoire | docker --version (démon actif) |
| Terraform                 | Obligatoire | terraform -version             |
| Éditeur de code (VS Code) | Obligatoire | —                              |
| AWS CLI                   | Optionnel   | aws --version                  |
| Azure CLI                 | Optionnel   | az --version                   |

Tableau 1 — Checklist logicielle

## 3.2 Checklist comptes cloud

- **GitLab** : Compte sur gitlab.com (ou instance fournie par l'établissement).
- **AWS** : Compte AWS avec accès IAM (création d'utilisateur avec accès programmatique et droits EC2 pour le TP).
- **Azure** : Compte Azure avec abonnement actif et Service Principal configuré (ARM\_CLIENT\_ID, ARM\_CLIENT\_SECRET, ARM\_SUBSCRIPTION\_ID, ARM\_TENANT\_ID).

## 3.3 Vérification des installations

Dans un terminal, exécuter les commandes de vérification listées ci-dessus. Pour Docker, s'assurer que docker run hello-world fonctionne correctement. Pour Terraform, exécuter terraform -version et vérifier qu'aucune erreur n'apparaît. Toute anomalie doit être résolue avant de poursuivre le TP.

## 4. Développement de l'application Node.js (Partie A)

L'objectif de cette partie est de créer une application web minimale avec Node.js et Express, exposant les routes / et /health. L'application utilise la variable d'environnement PORT pour la configuration du port d'écoute, et

suit une structure séparant l'application Express (src/app.js) du démarrage du serveur (src/server.js). Cette séparation facilite les tests unitaires en permettant d'importer l'application sans démarrer le serveur.

## 4.1 Procédure

Créer le dossier du projet (tp-devsecops-app), initialiser le projet avec `npm init -y`, installer Express avec `npm install express`, puis créer les fichiers `src/app.js` et `src/server.js`. Le fichier `package.json` doit définir `main`: `src/server.js` et le script `start`: `node src/server.js`. Créer `.env.example` avec `PORT=3000` et `.env` pour le développement local. Ajouter `node_modules`, `.env` et `*.log` au `.gitignore`.

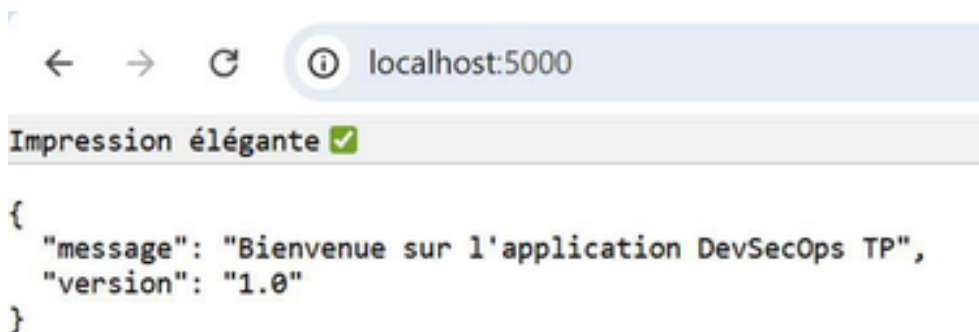


Figure 3 — Fichier `package.json`

## 4.2 Code source

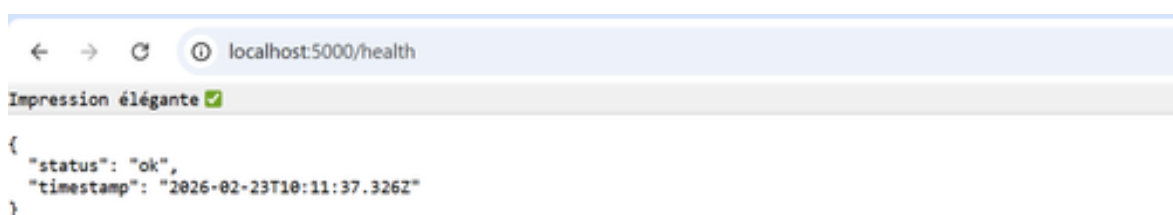


Figure 4 — Fichier `src/app.js`

La variable `process.env.PORT` permet de configurer le port sans modifier le code, ce qui est essentiel pour Docker et les environnements cloud. Le fichier `.env` n'est pas versionné pour éviter d'y stocker des secrets. Le résultat attendu est que l'application réponde en JSON sur `/` (message, version) et sur `/health` (status: ok, timestamp).

### 4.3 Erreurs fréquentes

| Problème                   | Solution                                                               |
|----------------------------|------------------------------------------------------------------------|
| Port déjà utilisé          | Changer PORT dans <code>.env</code> ou arrêter le processus            |
| Module introuvable         | Exécuter <code>npm install</code> depuis la racine du projet           |
| Cannot find module './app' | Vérifier que <code>src/app.js</code> existe et lancer depuis la racine |

Tableau 2 — Erreurs fréquentes (Partie A)

## 5. Tests automatisés avec Jest et Supertest (Partie B)

L'objectif de cette partie est de mettre en place des tests automatisés pour les routes `/` et `/health`, exécutables en local et dans le pipeline CI. Les tests sont écrits avec Jest comme framework de test et Supertest comme librairie de requêtes HTTP. Cette combinaison permet de tester les routes Express sans démarrer réellement le serveur, en utilisant l'objet `app` exporté depuis `src/app.js`.

### 5.1 Procédure

Installer Jest et Supertest en devDependencies : `npm install --save-dev jest supertest`. Ajouter le script test: `jest` dans `package.json`. Créer le dossier `__tests__` à la racine avec le fichier `app.test.js` contenant les tests GET `/` et GET `/health`. Lancer `npm test` pour vérifier que les tests passent.

### 5.2 Fichier de test

Le fichier `__tests__/app.test.js` contient deux tests : le premier vérifie que GET `/` retourne un statut 200 et un message contenant « DevSecOps », le second vérifie que GET `/health` retourne un statut 200 et un body avec status: « ok ». Ces tests garantissent le bon fonctionnement des endpoints critiques de l'application.

### 5.3 Échec volontaire puis correction

Pour valider le fonctionnement du pipeline de test, il est recommandé de modifier temporairement `src/app.js` (par exemple changer le message ou le status) pour faire échouer un test. Relancer `npm test` : le test concerné



doit être en échec. Restaurer ensuite le code correct et vérifier que les tests repassent. Cet exercice démontre l'utilité des tests automatisés pour détecter les régressions.

## 6. Conteneurisation avec Docker (Partie C)

L'objectif de cette partie est de produire une image Docker de l'application avec Node 20, de la construire et de l'exécuter en local pour valider le comportement en conteneur sur le port 3000. La conteneurisation garantit que l'application fonctionne de manière identique quel que soit l'environnement d'exécution, éliminant les problèmes de type « ça marche sur ma machine ».

### 6.1 Dockerfile

Le Dockerfile utilise l'image légère node:20-alpine, définit /app comme répertoire de travail, copie les fichiers de dépendances, exécute npm ci --only=production pour installer uniquement les dépendances de production, copie le code source, expose le port 3000, et définit la commande de démarrage. L'utilisation de npm ci plutôt que npm install garantit une installation déterministe.

### 6.2 Fichier .dockerignore

Le fichier .dockerignore exclut du contexte de build les éléments inutiles ou sensibles : node\_modules, .env, .git, \*.log, \_\_tests\_\_, .gitignore, README.md. Cette exclusion réduit la taille de l'image et évite d'inclure des fichiers sensibles dans le conteneur.

### 6.3 Commandes Docker

```
vsecops-app> docker build -t tp-devsecops-app:latest .
=> => extracting sha256:eb87f4721c91769ed5206f34a9ab6ec98fc1d5 0.1s
=> => extracting sha256:e31b2016552274339ed88ed4a438d78bf37e0f 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 198.35kB 0.1s
=> [2/5] WORKDIR /app 0.1s
=> [3/5] COPY package*.json ./ 0.0s
=> [4/5] RUN npm ci --only=production 2.4s
=> [5/5] COPY . . 0.1s
=> exporting to image 0.9s
=> => exporting layers 0.3s
=> => exporting manifest sha256:b0612b8ffe488c5f8ba3bdc16d89a2 0.0s
=> => exporting config sha256:c4d5bf6badd281b21e1b0ae021251da8 0.0s
=> => exporting attestation manifest sha256:5c760f0c05b835b5fa 0.0s
=> => exporting manifest list sha256:d70746461da196a2fc922cdc4 0.0s
=> => naming to docker.io/library/tp-devsecops-app:latest 0.0s
=> => unpacking to docker.io/library/tp-devsecops-app:latest 0.4s
```

Figure 7 — Commandes Docker de build et run



Figure 8 — Exécution du conteneur et vérification

## 6.4 Erreurs fréquentes

| Problème                             | Solution                                                  |
|--------------------------------------|-----------------------------------------------------------|
| Docker daemon non démarré            | Lancer Docker Desktop                                     |
| Erreur de build (npm ci)             | S'assurer que package-lock.json existe                    |
| Fichier non trouvé dans le conteneur | Vérifier que src/ est copié (pas exclu par .dockerignore) |

Tableau 3 — Erreurs fréquentes (Partie C)

## 7. Pipeline GitLab CI/CD : qualité et sécurité (Partie D+E)

L'objectif est de définir un pipeline GitLab avec les stages test, security, build, deploy\_staging et deploy\_production, puis de faire échouer le pipeline en introduisant un faux secret pour démontrer le contrôle de sécurité. Le pipeline constitue le cœur de l'approche DevSecOps en automatisant les vérifications de qualité et de sécurité à chaque push.

### 7.1 Structure du pipeline



Figure 9 — Définition des stages dans .gitlab-ci.yml

Le pipeline comprend cinq stages ordonnés : test (exécution des tests Jest), security (détection de secrets exposés par grep), build (construction et push de l'image Docker), deploy\_staging (déploiement automatique sur AWS via Terraform, branche develop), et deploy\_production (déploiement manuel sur Azure via Terraform, branche main).

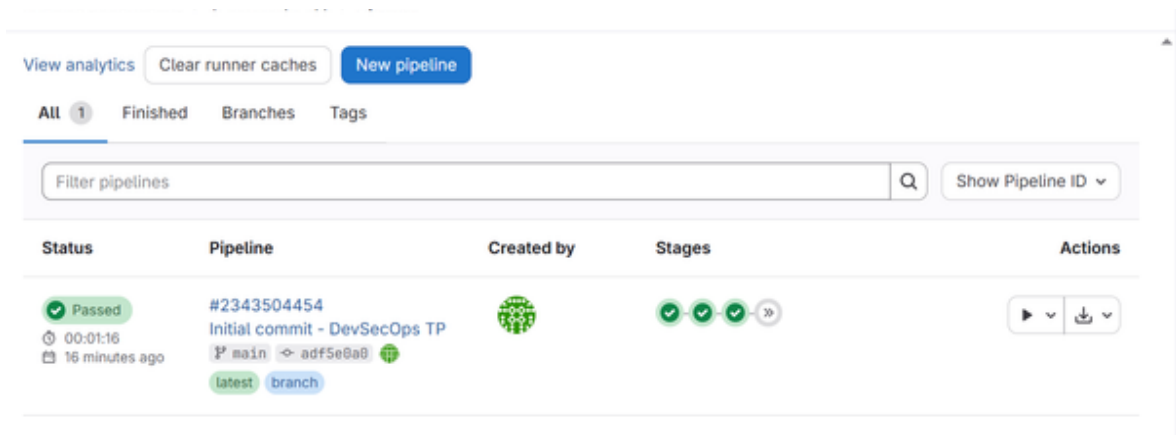


Figure 10 — Configuration du pipeline (stages test, security, build)

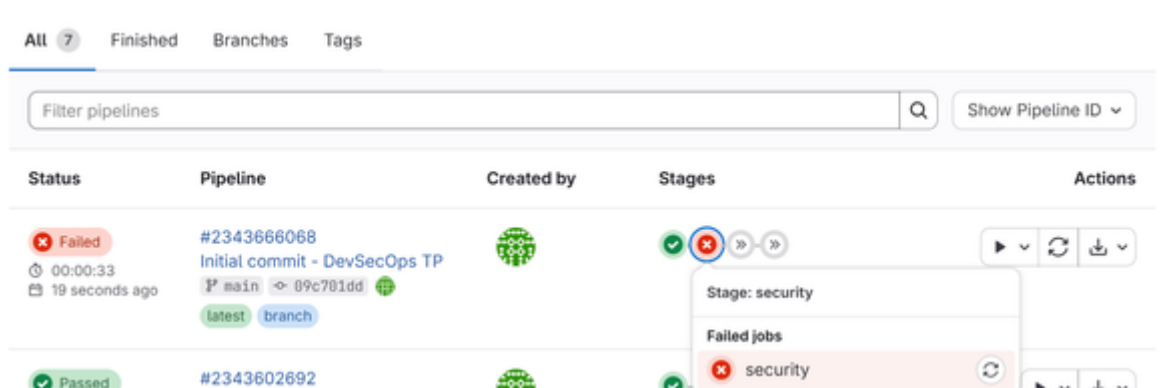


Figure 11 — Configuration du pipeline (stages deploy)

## 7.2 Stage test

Le stage test utilise l'image node:20-alpine, exécute npm ci pour installer les dépendances, puis lance npm test pour exécuter les tests Jest. Ce stage garantit que chaque commit ne introduit pas de régression fonctionnelle. Si

un test échoue, le pipeline s'arrête et les stages suivants ne sont pas exécutés, empêchant ainsi le déploiement d'une version défectueuse.

## 7.3 Stage security

Le stage security utilise l'image alpine:latest et exécute un grep récursif pour détecter les motifs de type `API_KEY=sk-...` dans les fichiers `.js` et `.json`. Si un secret est détecté, le job échoue (exit 1), bloquant le pipeline. Ce mécanisme simple illustre le principe du contrôle de sécurité automatisé dans le pipeline CI/CD. En production, ce type de contrôle serait complété par des outils spécialisés comme GitGuardian, TruffleHog ou gitleaks.

## 7.4 Stage build

Le stage build utilise l'image `docker:24` avec le service `docker:24-dind` (Docker-in-Docker). Il s'authentifie au GitLab Container Registry, construit l'image Docker avec deux tags (`$CI_COMMIT_SHORT_SHA` pour la traçabilité et `latest` pour le build le plus récent), puis pousse les deux tags vers le registry. Ce stage ne s'exécute que si les stages test et security ont réussi.

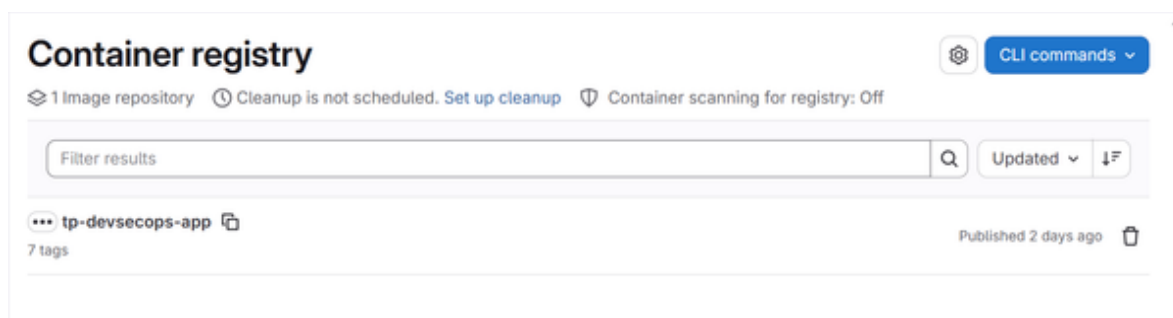


Figure 12 — Stage build : construction et push de l'image Docker

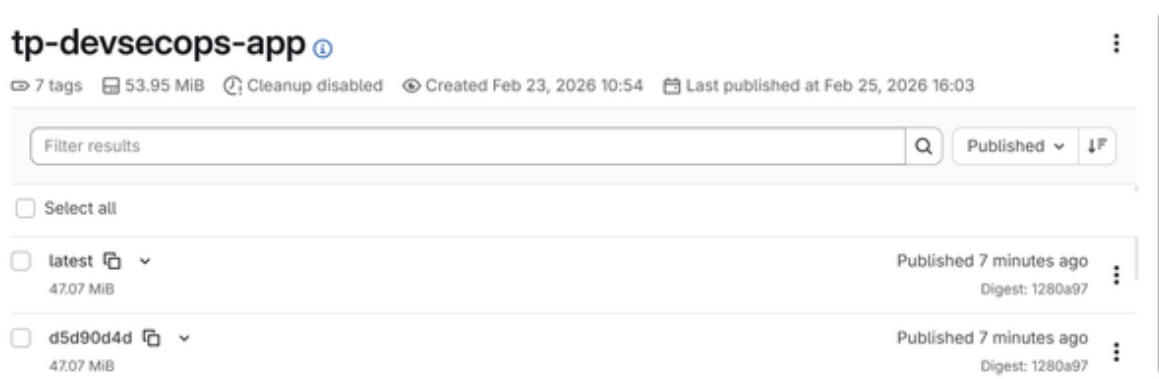


Figure 13 — Stages deploy\_staging et deploy\_production

## 7.5 Démonstration pédagogique : faux secret

Pour démontrer le fonctionnement du contrôle de sécurité, il convient d'injecter un faux secret dans un fichier source (par exemple `API_KEY=sk-demo-123456789`). Après commit et push, le job security échoue car le grep

trouve le motif. Il faut ensuite supprimer la ligne contenant le faux secret, la remplacer par l'usage d'une variable d'environnement (`process.env.API_KEY`), commiter et pousser à nouveau. Le pipeline repasse alors au vert. Il est impératif de n'utiliser que des faux secrets pour cet exercice.

## 8. Publication de l'image Docker dans GitLab Container Registry

Le job build du pipeline s'authentifie au GitLab Container Registry en utilisant les variables automatiquement fournies par GitLab : `CI_REGISTRY` (URL du registry), `CI_REGISTRY_IMAGE` (URL complète de l'image du projet), `CI_REGISTRY_USER` et `CI_REGISTRY_PASSWORD` (identifiants pour docker login). Après un push, le pipeline exécute le job build et l'image apparaît dans GitLab : Deploy → Container Registry avec les tags associés (SHA du commit, latest, et optionnellement des tags de version comme v1.0.0). Le Container Registry constitue le maillon central entre le pipeline de build et les environnements de déploiement.

## 9. Infrastructure as Code avec Terraform

Terraform permet de définir l'infrastructure cloud de manière déclarative, versionnable et reproductible. Les concepts clés sont les providers (modules communiquant avec un cloud), les ressources (éléments d'infrastructure), les variables (paramètres d'entrée), les outputs (valeurs exportées après apply), le state (fichier enregistrant l'état des ressources), et les commandes `plan/apply/destroy`.

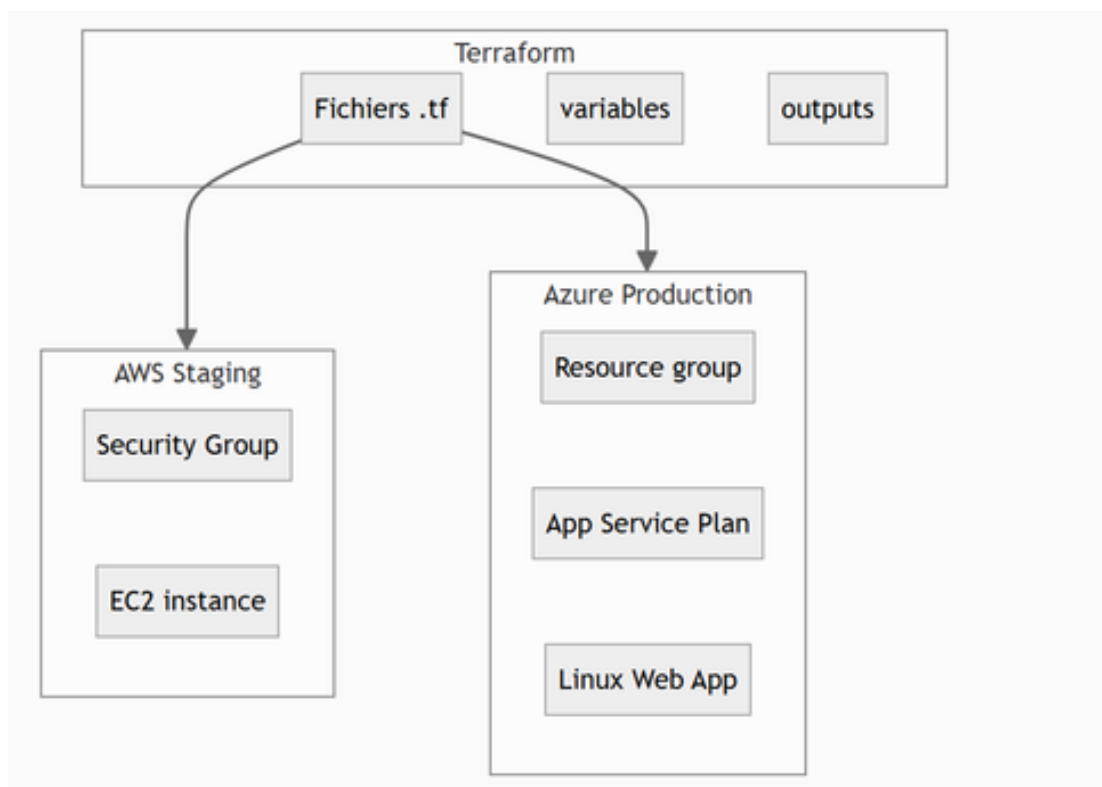


Figure 14 — Terraform multi-cloud (AWS staging / Azure production)

### 9.1 Arborescence imposée

| Chemin                                              | Description                                        |
|-----------------------------------------------------|----------------------------------------------------|
| terraform/aws-staging/main.tf                       | Configuration principale AWS (VPC, SG, EC2)        |
| terraform/aws-staging/variables.tf                  | Variables d'entrée AWS                             |
| terraform/aws-staging/outputs.tf                    | Valeurs exportées (IP publique)                    |
| terraform/aws-staging/terraform.tfvars.example      | Exemple de fichier de valeurs                      |
| terraform/azure-production/main.tf                  | Configuration principale Azure (RG, Plan, Web App) |
| terraform/azure-production/variables.tf             | Variables d'entrée Azure                           |
| terraform/azure-production/outputs.tf               | Valeurs exportées (URL production)                 |
| terraform/azure-production/terraform.tfvars.example | Exemple de fichier de valeurs                      |

Tableau 4 — Arborescence Terraform du projet

## 10. Déploiement en staging sur AWS (Partie F)

L'objectif est de créer l'infrastructure de staging sur AWS avec Terraform : VPC par défaut, Security Group (HTTP 80, SSH 22), instance EC2. Le script `user_data` installe Docker, se connecte au GitLab Registry, tire l'image et lance le conteneur sur le port 80 (mapping 80→3000). Cette approche permet de déployer automatiquement l'application lors de la création de l'instance, sans intervention manuelle.

### 10.1 Configuration Terraform AWS

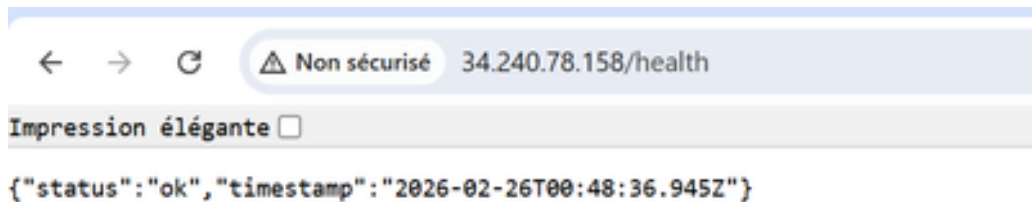


Figure 15 — Fichier main.tf AWS (instance EC2 avec user\_data)

Le fichier main.tf définit le provider AWS, le Security Group autorisant les ports 80 et 22, et l'instance EC2 avec un script user\_data. Ce script installe Docker au démarrage, s'authentifie au GitLab Container Registry avec les identifiants fournis via les variables Terraform, tire l'image Docker et lance le conteneur en mappant le port 80 vers le port 3000 de l'application.

## 10.2 Variables et outputs

Les variables Terraform pour AWS incluent aws\_region (défaut eu-west-1), instance\_type (défaut t2.micro), docker\_image, docker\_tag (défaut latest), registry\_url, registry\_user (sensitive) et registry\_password (sensitive). L'output staging\_public\_ip fournit l'adresse IP publique de l'instance pour tester l'accès à l'application via `http://<IP>/health`.

## 10.3 Erreurs fréquentes AWS

| Problème                             | Solution                                                 |
|--------------------------------------|----------------------------------------------------------|
| Port 80 inaccessible                 | Vérifier le Security Group (ingress 80 depuis 0.0.0.0/0) |
| Docker non lancé / image non trouvée | Se connecter en SSH et vérifier les logs                 |
| Registry login failed                | Vérifier le Deploy Token et registry_url (sans https://) |

Tableau 5 — Erreurs fréquentes (Déploiement AWS)

## 11. Déploiement en production sur Azure (Partie G)

L'objectif est de créer l'infrastructure de production sur Azure avec Terraform : Resource Group, App Service Plan, Linux Web App (App Service for Containers). L'application Azure tire l'image Docker depuis le GitLab Container Registry (image privée) et expose le port 3000 via WEBSITES\_PORT=3000. Azure App Service for Containers offre une gestion simplifiée des conteneurs avec auto-scaling, certificats SSL et domaines personnalisés.

## 11.1 Configuration Terraform Azure

```
m.tfvars"
data.aws_vpc.default: Reading...

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

tls_private_key.staging: Creating...
aws_security_group.staging_sg: Creating...
tls_private_key.staging: Creation complete after 3s [id=712c31d0865363
4ce39a97e922d466fbbaf59e2e]
aws_key_pair.staging: Creating...
local_file.private_key: Creating...
local_file.private_key: Creation complete after 0s [id=c5bc14aefdfa429
c6c006c871cb3385fe3088a3a]
aws_key_pair.staging: Creation complete after 1s [id=tp-staging-key]
aws_security_group.staging_sg: Creation complete after 4s [id=sg-0f185
1541515d97cd]
aws_instance.staging: Creating...
aws_instance.staging: Creation complete after 15s [id=i-058a6ae384fcd0
252]

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

staging_public_ip = "34.240.78.158"
```

Figure 16 — Fichier main.tf Azure (Resource Group, Service Plan, Web App)

Le fichier main.tf définit le provider Azure, un Resource Group, un Service Plan (SKU B1, Linux), et une Linux Web App configurée pour tirer l'image Docker depuis le GitLab Registry. Les app\_settings incluent WEBSITES\_PORT=3000, les identifiants du registry (DOCKER\_REGISTRY\_SERVER\_URL, USERNAME, PASSWORD) et la référence de l'image à déployer.

## 11.2 Erreurs fréquentes Azure



| Problème               | Solution                                                                            |
|------------------------|-------------------------------------------------------------------------------------|
| Image pull failed      | Vérifier l'URL du registry (avec https://), le nom d'utilisateur et le mot de passe |
| Port non exposé        | S'assurer que WEBSITES_PORT=3000 est défini dans app_settings                       |
| Authentification Azure | Vérifier les variables ARM_* et les droits du Service Principal                     |

Tableau 6 — Erreurs fréquentes (Déploiement Azure)

## 12. Chaîne de livraison : de GitLab à AWS/Azure (Partie H)

Cette section détaille pas à pas comment l'image Docker est construite, stockée, puis récupérée par AWS et Azure. Ce parcours illustre le rôle central du Container Registry comme point de passage obligé entre le pipeline de build et les environnements de déploiement.

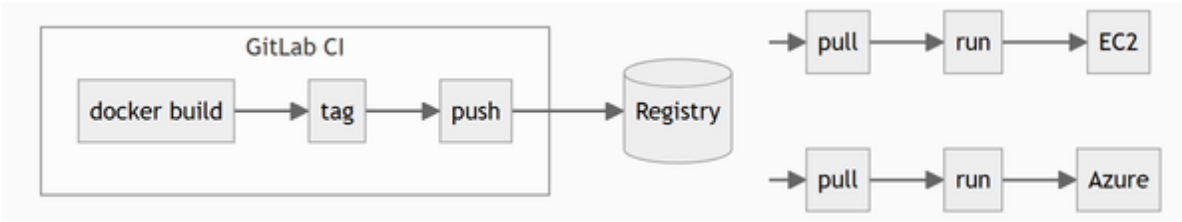


Figure 17 — Flux Docker (build / tag / push / pull / run)

### 12.1 Construction de l'image

Le runner GitLab CI construit l'image lors du stage build, après les stages test et security. L'image est construite à partir du Dockerfile à la racine du projet, avec les tags  
\$CI\_REGISTRY\_IMAGE:\$CI\_COMMIT\_SHORT\_SHA (pour la traçabilité) et  
\$CI\_REGISTRY\_IMAGE:latest (pour le build le plus récent). Cette stratégie de tagging garantit qu'à tout moment, il est possible d'identifier précisément quel commit a généré une image donnée.

### 12.2 Stockage dans le GitLab Container Registry

L'image est stockée dans le GitLab Container Registry du projet (Deploy → Container Registry). Les variables GitLab CI\_REGISTRY, CI\_REGISTRY\_IMAGE, CI\_REGISTRY\_USER et CI\_REGISTRY\_PASSWORD sont automatiquement fournies par GitLab dans chaque job CI. Les tags disponibles incluent

:SCI\_COMMIT\_SHORT\_SHA (traçabilité), :latest (dernier build) et optionnellement des tags de version (:v1.0.0).

### 12.3 Récupération par AWS et Azure

Pour AWS (staging), Terraform crée l’instance EC2. Au premier démarrage, le script user\_data installe Docker, se connecte au registry GitLab, télécharge l’image et lance le conteneur. Pour Azure (production), Terraform crée l’App Service qui tire automatiquement l’image depuis le GitLab Registry en utilisant les paramètres de connexion configurés dans azurerm\_linux\_web\_app. La variable WEBSITES\_PORT=3000 indique à Azure quel port le conteneur écoute.

### 12.4 Automatisation vs manuel

| Élément                | Manuel (TP)               | Automatisé (pipeline)          | En entreprise                 |
|------------------------|---------------------------|--------------------------------|-------------------------------|
| Tests                  | —                         | Job test à chaque push         | Idem + revue de code          |
| Sécurité               | —                         | Job security (grep secrets)    | Idem + scan dépendances, SAST |
| Build image            | docker build local        | Job build                      | Pipeline complet              |
| Push registry          | —                         | Job build (docker push)        | Pipeline automatisé           |
| Déploiement staging    | terraform apply à la main | Job deploy_staging (optionnel) | Automatisé sur develop        |
| Déploiement production | terraform apply à la main | Job deploy_production (manual) | Manual gate ou approbation    |

Tableau 7 — Manuel vs automatisé dans le TP et en entreprise

### 12.5 Qui fait quoi ?

| Acteur          | Rôle                                                                                |
|-----------------|-------------------------------------------------------------------------------------|
| Développeur     | Commit/push du code, déclenche le pipeline                                          |
| GitLab CI       | Exécute test, security, build ; login registry ; push image ; terraform apply       |
| Registry        | Stocke les images Docker, les rend disponibles pour AWS et Azure                    |
| Terraform       | Crée ou met à jour l'infrastructure (EC2, Security Group, Web App)                  |
| AWS (EC2)       | Exécute user_data : install Docker, login, pull, run ; expose sur port 80           |
| Azure (Web App) | Tire l'image depuis le registry ; exécute le conteneur ; expose selon WEBSITES_PORT |

Tableau 8 — Rôles des acteurs dans la chaîne de livraison

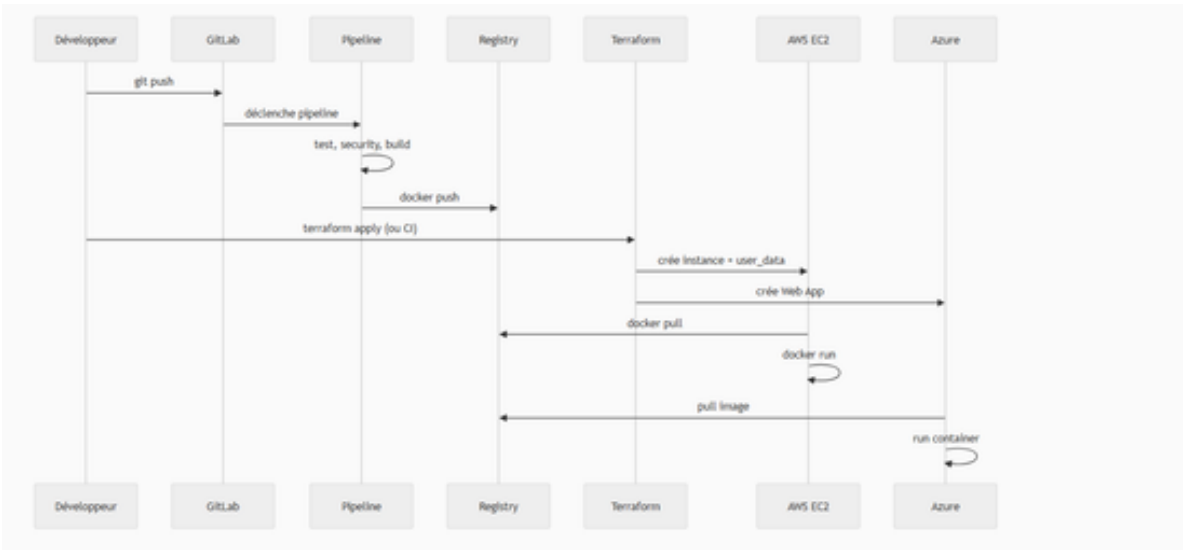


Figure 18 — Séquence complète : commit → pipeline → registry → Terraform → cloud

## 13. Validation finale et plan de démonstration (Partie I)

La validation finale consiste à vérifier l'ensemble de la chaîne DevSecOps, depuis l'application locale jusqu'aux environnements cloud. La checklist complète comprend six points de vérification : application locale (npm start, routes / et /health), tests (npm test passants), Docker local (build et run, routes accessibles), pipeline GitLab (stages test, security, build verts, image visible dans Container Registry), staging AWS (terraform apply, IP publique, /health accessible), et production Azure (terraform apply, URL, /health accessible).

### 13.1 Ordre de démonstration

La démonstration en classe suit un ordre logique : montrer le dépôt GitLab (structure, .gitlab-ci.yml), un pipeline réussi (Pipelines → jobs), le Container Registry (tags de l'image), l'URL staging AWS avec /health, l'URL production Azure avec /health, puis résumer en une phrase : « Le code est poussé → le pipeline build l'image et la met dans le registry → Terraform crée les serveurs → AWS et Azure tirent l'image et lancent le conteneur. »

## 14. Livrables à rendre

Les livrables attendus à l'issue du TP comprennent : le lien du dépôt GitLab (code source, .gitlab-ci.yml, terraform/), une capture des tests OK (npm test), une capture du pipeline sécurité KO (faux secret) puis OK (correction), une capture du Container Registry (image et tags), une capture du staging AWS (IP ou URL, réponse / et /health), une capture de la production Azure (URL, réponse / et /health), et un mini compte-rendu de 5 à 10 lignes résumant la chaîne de livraison et le rôle de chaque composant.

## 15. Dépannage rapide (FAQ)

Cette section recense les problèmes les plus fréquemment rencontrés lors du TP et propose des pistes de solution pour chacun. La consultation de cette FAQ est recommandée avant de demander de l'aide à l'encadrant.

| Problème                        | Piste de solution                                                                      |
|---------------------------------|----------------------------------------------------------------------------------------|
| npm install échoue              | Vérifier Node.js (node -v), supprimer node_modules et package-lock.json puis réessayer |
| Port déjà utilisé               | Changer PORT dans .env ou arrêter le processus                                         |
| Docker daemon ne démarre pas    | Lancer Docker Desktop ; sous Windows vérifier WSL2 ou Hyper-V                          |
| Job GitLab failed               | Lire les logs du job ; vérifier l'indentation YAML                                     |
| Secret détecté (pipeline rouge) | Supprimer toute ligne type API_KEY=sk-... ; utiliser process.env et GitLab Variables   |
| Terraform auth AWS              | Vérifier AWS_ACCESS_KEY_ID et AWS_SECRET_ACCESS_KEY ; droits IAM                       |
| Terraform auth Azure            | Vérifier ARM_CLIENT_ID, ARM_CLIENT_SECRET, ARM_SUBSCRIPTION_ID, ARM_TENANT_ID          |
| Image introuvable (pull failed) | Vérifier l'URL de l'image et l'accessibilité du registry                               |
| Site inaccessible               | Security Group (AWS) port 80 ; Azure WEBSITES_PORT=3000 ; conteneur lancé              |

Tableau 9 — FAQ Dépannage rapide

## 16. Nettoyage des ressources (coûts)

Pour éviter des facturations après le TP, il est impératif de détruire toutes les ressources créées. Pour AWS, exécuter terraform destroy dans terraform/aws-staging/ et confirmer par yes, puis vérifier dans la console AWS que l'instance EC2 et le Security Group ont été supprimés. Pour Azure, exécuter terraform destroy dans terraform/azure-production/ et confirmer par yes, puis vérifier dans le portail Azure que le Resource Group et la Web App ont été supprimés. Les ressources actives (VM, Web App) génèrent des coûts tant qu'elles existent.

